Cahier des charges
Projet informatique individuel
Cogmeetic

SCHNEIDER ARTHUR
2A GROUPE 2

1. Contexte et objectifs

Chaque année, l'ENSC met en place un système de parrainage entre les élèves de première année (1A) et ceux de deuxième année (2A). Actuellement, les couples parrain·e / filleul·e sont constitués à partir de formulaires, de fichiers Excel et d'échanges informels entre membres des associations étudiantes. Cette organisation fonctionne, mais elle est longue à gérer, peu automatisée et offre une visibilité limitée aux élèves sur la façon dont les couples sont créés.

Le projet Cogmeetic vise à proposer une application web dédiée pour accompagner ce parrainage. L'objectif principal est de rendre la constitution des binômes plus fluide, plus transparente et plus ludique, en s'appuyant sur des mini-jeux de type « Tu préfères… », sur des profils simplifiés et sur un calcul de compatibilité entre élèves. L'application doit permettre de recueillir des préférences, de suggérer un Top 5 de personnes avec qui le courant semble bien passer, et de fournir à l'équipe organisatrice une base de couples exploitable pour le parrainage réel.

Sur le plan pédagogique, Cogmeetic est mené dans le cadre du module P2I comme un projet individuel. Il doit permettre de partir d'une maquette existante, de formaliser un cahier des charges, de définir une architecture front-end / back-end cohérente, de développer un prototype fonctionnel et de préparer un livrable final accompagné d'une démonstration.

2. Acteurs et besoins

Les principaux utilisateurs sont les élèves de 1A et de 2A, qui participent au dispositif en tant que filleul·es ou parrains/marraines. Ils doivent pouvoir créer un compte, se connecter à l'application, disposer d'un espace personnel et compléter un profil simple (nom, promo, biographie courte, centres d'intérêt, moyens de contact). Ils interagissent avec un mini-jeu de questions « Tu préfères… » qui enregistre leurs réponses et alimente un score de compatibilité. Ils peuvent consulter une liste de personnes de la promo opposée triée selon cette compatibilité et constituer un Top 5 de profils avec lesquels ils aimeraient être associés pour le parrainage. L'application sert également de support à des échanges, via une messagerie simplifiée qui permet de discuter avec certains profils jugés compatibles.

L'équipe organisatrice (par exemple le BDF ou l'association en charge du parrainage) utilise l'application dans une logique de supervision. Elle doit pouvoir visualiser les couples proposés par l'algorithme, repérer les élèves non appariés ou les déséquilibres (par exemple une même personne demandée par trop de monde), et récupérer facilement la liste finale des couples pour la communication officielle. Dans cette première version, l'accent est mis sur l'affichage d'une liste de couples calculés à partir des Top 5 et des compatibilités, plutôt que sur des fonctionnalités complètes d'édition et de validation manuelles.

3. Fonctionnalités principales

L'application commence par un parcours d'authentification simple. Une page d'inscription permet à un nouvel utilisateur de choisir un pseudo, de définir un mot de passe et d'indiquer sa promo (1A ou 2A). Une page de connexion lui permet ensuite de retrouver son compte à partir de ces identifiants. Dans le prototype, ces opérations sont simulées côté serveur, avec un petit ensemble d'utilisateurs de test, mais le parcours est conçu comme s'il pouvait être utilisé par une vraie promotion.

Après l'inscription, un court tutoriel d'onboarding s'affiche. Il présente le principe de Cogmeetic, rappelle qu'il s'agit d'un outil de parrainage et non d'une application de rencontre, explique le fonctionnement des questions quotidiennes et du Top 5, et insiste sur la nécessité de rester bienveillant dans les échanges. Ce tutoriel n'est montré qu'une fois par utilisateur ; ensuite, celui-ci est redirigé directement vers la page principale.

La page centrale est celle du jeu « Tu préfères… ». L'élève voit une carte au milieu de l'écran et choisit entre deux options en la faisant glisser vers la gauche ou vers la droite. Chaque swipe correspond à une réponse envoyée au serveur, associée à l'utilisateur et à la question concernée. Pour cette première version, trois questions sont proposées par jour et par utilisateur. Une fois les trois questions répondues, un écran indique que la session du jour est terminée et invite l'élève à revenir plus tard. Les réponses ainsi enregistrées servent de base au calcul de compatibilité entre élèves de promos différentes.

La compatibilité est définie de manière volontairement simple : pour deux utilisateurs A et B, on compare les questions auxquelles ils ont tous les deux répondu, et l'on calcule le ratio entre le nombre de réponses identiques et le nombre total de questions communes. Une route de l'API permet, à partir de l'identifiant d'un utilisateur, de récupérer la liste des élèves de la promo opposée accompagnés de ce pourcentage de compatibilité. Ces valeurs sont ensuite utilisées pour trier les listes et pour préremplir, si souhaité, un Top 5 automatique.

La page « Poulpage » permet à chaque élève de classer jusqu'à cinq personnes de la promo opposée. L'interface affiche cinq positions, de 1 à 5, et l'utilisateur peut choisir qui placer à chaque rang. Il peut le faire manuellement, ou bien activer un mode dans lequel l'algorithme remplit automatiquement ces rangs avec les profils ayant les meilleurs scores de compatibilité actuels. Une fois le Top 5 validé, l'information est stockée côté serveur et servira plus tard à un algorithme de matching qui favorisera, par exemple, les couples où deux personnes se sont mutuellement placées en tête de leur classement.

Chaque utilisateur dispose aussi d'une page de profil. Elle présente ses informations principales et peut être consultée en lecture seule. Un bouton permet d'entrer dans un mode édition, de modifier la bio, les centres d'intérêt et les contacts, puis d'enregistrer ces changements. L'objectif est de fournir un contexte rapide aux autres élèves lorsqu'ils consultent un profil depuis la page de compatibilité, de chat ou de Top 5.

Une page « Événements » complète l'ensemble. Elle affiche une liste d'événements (soirées, activités, week-ends, etc.) fournis par l'API, avec un visuel, un titre, une date, un créneau horaire et un prix indicatif. Dans cette première version, cette page est informative : elle relie les interactions en ligne à des occasions de se rencontrer réellement, mais ne gère pas encore les inscriptions. Enfin, une vue administrateur permet d'afficher les couples 1A / 2A proposés par l'algorithme,

accompagnés de quelques indicateurs (compatibilité, rangs dans les Top 5). Cette vue sert de base de travail à l'équipe organisatrice pour préparer la liste finale des binômes.

4. Contraintes techniques et organisation du travail

Sur le plan technique, le front-end est développé en React, avec Vite pour l'initialisation et le bundling du projet. La navigation entre les différentes pages (connexion, inscription, tutoriel, jeu, Top 5, profil, chat, événements, administration) est gérée par React Router. Le style est assuré par des fichiers CSS organisés par page, en s'appuyant sur les couleurs et les dégradés définis dans la maquette Figma. L'interface est pensée dès le départ pour être responsive et utilisable sur smartphone.

Le back-end est réalisé en Node.js avec Express. Il expose une API REST qui gère la connexion/inscription simulée, la fourniture des questions du jour, l'enregistrement des réponses, le calcul de compatibilité, la lecture et l'écriture des Top 5, la consultation et la mise à jour des profils, l'accès aux événements et la génération des couples pour la vue administrateur. Les données sont stockées en mémoire dans cette première itération, ce qui permet de se concentrer sur la logique fonctionnelle.

Le projet étant individuel, l'étudiant assure l'ensemble des tâches : rédaction du cahier des charges, choix techniques, développement du front-end et du back-end, intégration, tests et préparation de la soutenance. Le travail est rythmé par les jalons du module P2I (cahier des charges, livrable intermédiaire, livrable final), qui servent de points de contrôle pour présenter l'avancement, ajuster le périmètre si nécessaire et s'assurer que le prototype reste réalisable dans le temps imparti.

Planning prévisionnel:

étapes	semaines													
	janvier		fevrier				mars				avril			
	19/01	26/01	02/02	09/02	16/02	23/02	02/03	09/03	16/03	23/03	30/03	06/04	13/04	20/04
backlog + organisation (Git / rôles / outils)														
Reprise de la maquette → design system + structure pages/routes														
Setup front (React/Vite), routing, layout, composants UI de base														
Écrans principaux (liste / détail / formulaires)														
Validation formulaires + messages erreur/loading + responsive														
Setup API + base de données + modèles + migrations/seed														
Endpoints CRUD principaux (ressources métier)														
Connexion front ↔ API (services),														
Scénario complet														
Cas erreurs + sécurité minimale														
Tests (min) + doc (README) + packaging														
Stabilisation / corrections / améliorations UX														